

SweetyStoryLab

Architecture and Development Guide

SweetyStoryLab Engineering

v1.1 — Multi-Brand Validation Release (updated from v1.0.0)

1. Executive Summary
 - What SweetyStoryLab Is
 - Goals of the Project
 - What v1.0 Accomplished
 - What This Update (v1.1) Adds
2. Project Timeline
3. Architecture Overview
 - The Core Principle
 - Module Responsibilities
4. Design Philosophy
 - A New Design Tension, Surfaced and Resolved: When *Is* Sharing Code the Right Call?
5. Folder Structure
6. End-to-End Pipeline Walkthrough
7. Configuration Reference
8. Providers
9. Brand Development Guide
10. Major Debugging Journey
 - 10.7 — The “correct” FFmpeg parameter that did nothing (empirical calibration)
 - 10.8 — Two unbounded FFmpeg inputs feeding one complex filtergraph
11. Important Design Decisions
12. Future Improvements (Not Yet Implemented)
13. Lessons Learned

1. Executive Summary

What SweetyStoryLab Is

SweetyStoryLab is a reusable AI storytelling platform, not a single automated Facebook page. It is a **Content Factory**: one shared codebase capable of generating, assembling, and publishing short-form video content across multiple storytelling “brands” (also called “Universes”) — Horror Lab and Mystery Lab today, with Bible Stories, History Files, Animal Stories, and others planned.

The defining architectural rule of the platform is this: **the engine must never know what genre it is generating**. Every brand supplies its own configuration, prompts, branding, voice mapping, fallback content, and Facebook credentials. The engine (`core/`) only ever knows about *pipeline stages* — generate a story, render segments, generate narration, assemble a video, publish, log — never about horror, mystery, or any other genre’s content.

Goals of the Project

- **Modularity** — each responsibility is isolated behind its own interface.
- **Maintainability** — a bug fix or improvement to core mechanics benefits every brand simultaneously.
- **Scalability** — designed against “will this still work with 50

brands?”, not just the two currently in scope.

- **Provider independence** — every external service sits behind an abstract interface, swappable without touching brand code.
- **Configuration over hardcoding** — brand identity lives entirely in data, never hardcoded into the engine.

What v1.0 Accomplished

v1.0 was the extraction and validation of Horror Lab — the platform’s original production brand — out of a monolithic script into the modular `core/` + `brands/horror_lab/` architecture. It was tagged after a full reconciliation against the real production script, a formal architecture sign-off review, a release-readiness audit that caught a release-blocking dynamic-import bug, and a live end-to-end run that published a real Facebook Reel.

What This Update (v1.1) Adds

Since v1.0, the platform has been genuinely stress-tested as a *platform*, not just validated as a single-brand extraction:

- **Mystery Lab shipped as the second brand** — a completely independent editorial identity (documentary/wonder-driven, not horror), added entirely through `brands/mystery_lab/` with **zero changes to `core/`**. This is the actual proof of the platform’s founding claim, not just an enabling capability.
- **A shared adaptive text-layout system** (`core/renderers/text_layout.py`) was extracted, giving both brands cinematic, auto-sizing hero titles and closing-question treatments — visual quality neither brand had in v1.0.
- **Per-brand visual identity became fully config-driven**, closing a gap the v1.0 architecture review had only partially addressed: question-slide font size and text color are now brand-config values (`question.max_font_size`, `question.text_color`), not hardcoded shared constants — the same “config, not branching” fix pattern applied a third and fourth time.
- **The burned-in subtitle treatment was substantially reworked** — a soft cinematic bottom gradient replaced a heavy “meme-style” outline, requiring a real, documented empirical calibration of an undocumented libass scaling behavior (see §10.7).
- **A developer testing convenience**, `--count N`, was added to the CLI, letting either brand generate N independent stories locally without publishing, each saved to its own inspectable folder — purpose-built for exactly the kind of “does the model actually generate different content” question that comes up constantly during prompt iteration.

None of this required touching the core/brand boundary established in v1.0. If anything, every addition above reinforced it — each new visual customization request was resolved by extending config, not by writing a brand-aware conditional anywhere in `core/`.

2. Project Timeline

(Phases 0–9, covering the original Horror Lab extraction and v1.0 release, are unchanged from the original document — reproduced in full below for continuity, followed by the phases added since.)

Phase 0 — The starting point. A single working script, `post.py`, already live in production, automating Horror Lab’s Facebook Reels — Groq, Pollinations, Edge TTS/ElevenLabs, FFmpeg, Facebook Graph API, SQLite, all orchestrated by GitHub Actions cron. Genre, brand identity, and engine logic were all fused into one file.

Phase 1 — Vision and platform framing. Reframed from “automate Horror Lab” to “build a platform that can run any number of storytelling brands from one engine,” tested against: “*Will this still work if there are 50 brands?*”

Phase 2 — Architectural design (no code). The core/brand boundary defined on paper: core owns behavior, brands own identity. A generic `Story/Segment` model designed for arbitrary, brand-defined segment sequences.

Phase 3 — First extraction pass. `post.py` split into `core/` + `brands/horror_lab/`, built from an initial reading of the repository — later found to contain real inaccuracies, including entirely fabricated functionality.

Phase 4 — Real-file reconciliation. The actual `post.py` diffed line-by-line; every fabrication corrected. Produced `ASSUMPTION_AUDIT.md`.

Phase 5 — Two deliberate architectural decisions. `narrator_gender` generalized to an opaque `voice_profile` string; fallback resilience split into a core mechanism (`FallbackProvider`) with brand-owned content.

Phase 6 — Architecture sign-off review. Found and fixed a recurring pattern of core silently defaulting to Horror Lab’s exact values, plus a previously-unverified caption-formatting bug.

Phase 7 — v1.0 release audit. Found the project’s single most serious defect — a dynamic-import path bug making the pipeline unable to run at all — invisible to every prior static check.

Phase 8 — Live validation. Real credentials, real local Windows environment. Surfaced three further real bugs: an FFmpeg subtitle-path escaping failure, a working-directory regression from that fix, and an asset-path consistency gap.

Phase 9 — v1.0 tagged. First successful real, live Facebook Reel publish. Release `v1.0.0`: modular Story Engine with first successful Facebook Reel.

Phase 10 — Mystery Lab brand identity design. Before any code, a full brand identity exercise: mission, editorial principles (“wonder over fear,” explicit fact/theory separation, “The One Question” litmus test), visual direction (archival/documentary, muted sepia/parchment palette), and a locked tagline (“History still has secrets.”), captured in an internal Brand Manifesto document — deliberately kept separate from marketing copy.

Phase 11 — Mystery Lab implementation, validating the platform claim. `brands/mystery_lab/` built component by component — system/user prompts translating the manifesto into structural generation requirements (not just tone guidance), an independent `MysteryLabParser` (deliberately *not* sharing code with `HorrorLabParser`, per an explicit decision that brand isolation outweighed a small amount of mechanical duplication at this stage), a `MysteryLabFallbackProvider` whose canonical fallback story (the Voynich Manuscript) was chosen specifically for long-term editorial safety — no real victims, genuinely durable “still unsolved” status — and a `config.yaml` proving the

existing schema needed zero new keys. **Zero core changes were required at any point**, confirmed by direct verification, not assumption.

Phase 12 — Hero title renderer redesign. The narration frame’s title treatment was rebuilt from a small, character-truncated single line into an adaptive, cinematic hero title: real pixel-width-measured balanced line wrapping, a font size chosen by searching for the largest size that fits a defined region, and the whole title/divider/scene-dot group bottom-anchored as one composed unit rather than independently centered elements.

Phase 13 — Question slide redesigned as the emotional climax. Following an explicit design discussion (analyzing a visual reference for principles worth adopting vs. avoiding, given “wonder over fear” and “no grunge/clickbait styling”), the closing question slide was rebuilt to feel like a documentary’s closing title card: a higher font ceiling than narration titles, the only gold-colored text in the video, and — after a real measured defect (over 1000px of unintended empty space above short questions) — a **group-centered**, not bottom-anchored, composition, since the question’s available canvas space is much larger relative to its text than the narration hero region’s.

Phase 14 — Shared adaptive text-layout extraction. The wrap/fit algorithm built for the hero title was extracted into `core/renderers/text_layout.py` once the question slide needed the identical generic capability — explicitly distinguished from the earlier decision to keep Horror Lab’s and Mystery Lab’s *parsers* separate: that was two brands owning similar-but-independent brand logic; this was one core subsystem needing the same generic algorithm twice.

Phase 15 — Subtitle treatment overhaul, and a real empirical-calibration detour. The burned-in narration captions were reworked from a heavy “meme subtitle” recipe (thick outline, bold, `FontSize=9`) to a lighter, more cinematic style, paired with a soft, restrained bottom gradient. Implementing the gradient required unifying the video pipeline’s FFmpeg filter graph into a single `-filter_complex`, which surfaced two real, non-obvious bugs (see §10.7, §10.8) — including the discovery that a “correct,” documented FFmpeg parameter (`original_size`) had **zero actual effect** on the specific scaling behavior being fixed, requiring a deliberate, explicitly-documented empirical calibration instead.

Phase 16 — Per-brand visual identity generalized via config, twice more. Two separate requests — reducing Horror Lab’s question font size without affecting Mystery Lab, then restoring Horror Lab’s white question text against Mystery Lab’s gold — each required promoting a previously-shared, hardcoded renderer constant into brand config (`question.max_font_size`, `question.text_color`), following the exact pattern already established for `font_path`, `watermark_text`, and `image_style_suffix`, with zero brand-aware branching introduced into the renderer.

Phase 17 — `--count` developer testing feature. Added to `core/pipeline/cli.py` and `core/pipeline/run.py`: generates N independent stories per invocation, never publishes, saves each to its own numbered folder (`output/<brand>/<timestamp>/story_NN/`) containing the full generated story JSON, images, video, subtitles, and metadata. Verified — using mocked network calls but the *real* story engine, parser, and renderers — to produce genuinely distinct content across iterations, with no caching or state leakage.

3. Architecture Overview

The Core Principle

Unchanged from v1.0, and now proven rather than merely designed: `core/` is genre-agnostic. It has no knowledge of Horror Lab, Mystery Lab, or any brand's content — confirmed repeatedly by direct repo-wide grep audits throughout this document's later phases, not just asserted once at release.

Module Responsibilities

(All v1.0 module descriptions remain accurate; one new module has been added.)

`core/renderers/text_layout.py` (new since v1.0) — Shared adaptive text-fitting machinery: `text_width()`, `wrap_balanced()` (pixel-measured, balanced multi-line wrapping, not character-count truncation), `fit_text()` (searches for the largest font size, within caller-supplied bounds, whose wrapped block fits a given box), and `draw_archival_divider()` (a shared thin-rule-with-diamond motif). Used identically by `narration_scene.py` and `question_slide.py`. This is a deliberate exception to “don't share code prematurely” — justified because it's one algorithm needed twice by *core itself*, not two brands independently owning similar logic.

`core/renderers/narration_scene.py` / `question_slide.py` — Updated since v1.0 to consume `text_layout.py`'s shared helpers instead of ad hoc truncation, and to accept brand-supplied visual parameters (`question_max_font_size`, `question_text_color`) via `RenderContext`, falling back to brand-neutral module defaults when a brand doesn't override them.

`brands/mystery_lab/` (new since v1.0) — Same shape as `brands/horror_lab/`: `config.yaml`, `prompts/system_prompt.txt` + `user_prompt.txt`, `parser.py` (`MysteryLabParser`), `fallback.py` (`MysteryLabFallbackProvider`), `assets/`. Deliberately independent from Horror Lab's equivalent files at every layer — no shared brand-level helper code, per the explicit brand-isolation decision in Phase 11.

4. Design Philosophy

All five v1.0 principles (separation of concerns, dependency direction, brand isolation, provider abstraction, configuration-driven design) hold unchanged, and each was independently re-tested by the work since v1.0 rather than merely re-asserted:

- **Brand isolation** was the literal subject of Mystery Lab’s entire implementation (Phase 11) — the platform’s founding claim was directly tested, not just designed for.
- **Configuration-driven design** was reinforced twice more (Phase 16): both times a *visual*, not just mechanical, brand distinction (font size, text color) was requested, the fix was “add a config field,” never “add a conditional.”
- **The composition root pattern** (`run_brand()` as the sole resolver of brand-relative values into concrete objects) was extended cleanly to two new config values (`question_max_font_size`, `question_text_color`) with no change to the pattern itself.

A New Design Tension, Surfaced and Resolved: When *Is* Sharing Code the Right Call?

v1.0 established a strong bias against premature sharing (the Horror Lab/Mystery Lab parser duplication decision). Phase 14 required drawing a real distinction, not just repeating the same answer: **duplication across brands is preserved deliberately (isolation has more value than avoiding a small amount of mechanical repetition); duplication of a generic algorithm *within core itself*, needed by two renderer files for identical technical reasons, is not the same category of problem, and was extracted.** The test applied: does the duplicated thing represent brand *identity* (keep separate) or generic *mechanism* (share)? `text_layout.py`’s wrap/fit logic is unambiguously mechanism.

5. Folder Structure

```
sweetystorylab/
├── core/
│   ├── config/
│   │   ├── loader.py
│   │   └── platform.py
│   ├── story/
│   │   ├── schema.py
│   │   ├── parser_base.py
│   │   ├── fallback_base.py
│   │   └── engine.py
│   └── renderers/
│       ├── registry.py
│       └── text_layout.py          # NEW -- shared adaptive wrap/fit + di
├── vider
│   ├── narration_scene.py
│   ├── question_slide.py
│   ├── providers/
│   │   ├── llm/
│   │   ├── image/
│   │   ├── tts/
│   │   ├── music/
│   │   └── publish/
│   └── video/
│       └── assembler.py          # UPDATED -- gradient, lighter subtitl
├── es
│   ├── storage/
│   │   └── log_store.py
│   └── pipeline/
│       └── run.py                # UPDATED -- run_brand_batch() for --c
├── ount
│   └── cli.py                   # UPDATED -- --count flag
├── brands/
│   ├── horror_lab/
│   │   └── config.yaml          # UPDATED -- question.max_font_size/te
├── xt_color
│   ├── prompts/
│   ├── parser.py
│   ├── fallback.py
│   ├── assets/
│   └── mystery_lab/            # NEW -- the second brand
│       ├── config.yaml
│       ├── prompts/
│       │   ├── system_prompt.txt
│       │   └── user_prompt.txt
│       └── parser.py            # MysteryLabParser -- independent from
├── Horror Lab's
│   └── fallback.py              # MysteryLabFallbackProvider (Voynich
├── Manuscript)
│   └── assets/
│       ├── fonts/
│       └── sounds/
├── .github-workflows-horror_lab-post.yml
├── ASSUMPTION_AUDIT.md
└── RELEASE_AUDIT.md
```

6. End-to-End Pipeline Walkthrough

Unchanged in structure from v1.0 (§6 of the original document remains accurate step-by-step) with two additions:

Step 6 (per-segment rendering) now resolves `question_max_font_size` and `question_text_color` from brand config into `RenderContext` before dispatching to `question_slide.py`, alongside the already-existing `font_path/watermark_text`.

A new invocation mode exists alongside the normal single-story path: `python -m core.pipeline.cli --brand <id> --count N` routes to `run_brand_batch()` instead of the normal publish-or-dry-run flow — it calls `run_brand()` in a loop with publishing forcibly disabled (`force_dry_run=True`, regardless of whether real credentials exist), copies each run's actual output files into `output/<brand>/<timestamp>/story_NN/`, and writes `story.json + metadata.json` per story. Omitting `--count` leaves the original single-story behavior completely unchanged.

7. Configuration Reference

Every field currently read from `brands/<id>/config.yaml`, who owns it, and where it's consumed.

Field	Owner	Consumed by
<code>brand.id</code> , <code>brand.name</code>	Brand	<code>core/config/loader.py</code> (identity); used as <code>brand_id</code> throughout the pipeline and logging
<code>brand.emoji</code> , <code>brand.parent</code>	Brand	Loaded and validated, not consumed by pipeline logic — descriptive metadata only
<code>content.system_prompt_file</code> , <code>content.user_prompt_file</code>	Brand	<code>run.py</code> — read as raw text, passed to the LLM provider
<code>content.parser_module</code>	Brand	<code>run.py</code> — dynamically imported to construct the brand's <code>StoryParser</code>
<code>content.fallback_provider_module</code>	Brand	<code>run.py</code> — dynamically imported to construct the brand's <code>FallbackProvider</code>
<code>content.segment_template</code>	Brand	Documents intended shape; actual sequence comes from parser output (see §12 for planned enforcement)
<code>llm.model</code> , <code>llm.temperature</code> , <code>llm.max_tokens</code>	Brand	<code>run.py</code> → <code>GroqProvider + generate_story()</code> . Required — no core-level default
<code>voice.profiles</code> , <code>voice.default_profile</code>	Brand	<code>run.py</code> — resolves <code>Story.voice_profile</code> to TTS settings. Validated at load time
<code>branding.watermark_text</code>	Brand	<code>run.py</code> → <code>RenderContext</code> → both renderers
<code>branding.font</code>	Brand	<code>run.py</code> — brand-relative path via <code>os.path.join(brand_dir, config.font_file)</code>
<code>branding.font_source_url</code>	Brand (docs only)	Not read by Python — informs the brand's GitHub Actions workflow
<code>music.local_file</code>	Brand	<code>run.py</code> — brand-relative path, resolved same way as <code>branding.font</code>
<code>image.style_suffix</code>	Brand	<code>run.py</code> → <code>PollinationsProvider</code> — appended to every image prompt
<code>image.fallback_dir</code>	Brand	<code>PollinationsProvider</code> — local fallback image directory
<code>image.fallback_color</code>	Brand	<code>run.py</code> 's exception handler — last-resort frame color
<code>question.max_font_size</code> (new)	Brand	<code>run.py</code> → <code>RenderContext</code> → <code>question_slide.py</code> . Brand-neutral default: 117pt
<code>question.text_color</code> (new)	Brand	<code>run.py</code> → <code>RenderContext</code> → <code>question_slide.py</code> . Brand-neutral default: white
<code>facebook.page_id_env</code> , <code>facebook.access_token_env</code>	Brand	<code>core/config/loader.py</code> — env var <i>names</i> to resolve, not secrets themselves
<code>schedule.times_pht</code>	Brand (docs only)	Not read by Python — scheduling owned entirely by the GitHub Actions cron

		trigger
--	--	---------

Platform-level (not in any brand’s config.yaml):

Variable	Owner	Consumed by
GROQ_API_KEY	Platform	core/config/platform.py
ELEVENLABS_API_KEY	Platform (optional)	core/config/platform.py

Current per-brand values, for reference:

Field	Horror Lab	Mystery Lab
question.max_font_size	110pt	117pt
question.text_color	White (255,255,255)	Gold (198,156,84)
branding.watermark_text	“SweetyStoryLab”	“Mystery Lab”

8. Providers

Unchanged from v1.0 — `LLMProvider`, `ImageProvider`, `TTSPProvider`, `MusicProvider`, `PublishProvider`, each with exactly one implementation today, each swappable without touching brand code. No new providers were added since v1.0; the work since then extended *rendering* and *pipeline orchestration*, not the provider layer.

9. Brand Development Guide

The steps are unchanged from v1.0 — and are no longer a theoretical description of an “extension test,” but a literal record of how Mystery Lab was actually built (Phase 11), confirmed to have required zero `core/` changes.

One addition worth calling out explicitly, learned directly from Mystery Lab’s build: **decide your brand’s visual identity (colors, font-size feel) as part of brand design, not as an afterthought** — Horror Lab’s question-slide color and size had to be retrofitted into config after the fact (Phase 16), which worked cleanly *because* the config-driven pattern already existed, but a new brand today can declare `question.max_font_size` and `question.text_color` from day one rather than inheriting shared defaults it may not want.

10. Major Debugging Journey

Sections 10.1–10.6 are unchanged from the original document (dynamic import path failure, Windows FFmpeg subtitle path parsing, asset path ownership regression, local vs. CI testing, Facebook authentication, runtime validation gaps). Two new entries follow, from the subtitle/gradient work.

10.7 — The “correct” FFmpeg parameter that did nothing (empirical calibration)

Root cause: FFmpeg’s `subtitles` filter, given a plain `.srt`, converts it to ASS internally using its own default script resolution (commonly 384×288 in libass) — **not** the actual output video resolution. This meant `MarginV` in the subtitle style string was silently being scaled by roughly $6.8\times$, not interpreted as literal pixels.

Investigation: the documented fix for exactly this class of problem, FFmpeg’s `original_size` filter option, was applied and *measured* — by rendering real frames and locating the actual subtitle pixel position via image analysis, not by trusting the documentation. The measurement showed **zero effect**: the scaling ratio was identical before and after adding the parameter. `original_size` is documented as an aspect-ratio/font-scaling aid, not a script-resolution override, and doesn’t apply to this code path.

Final solution: rather than build real `.ass`-file generation/rewriting infrastructure to force literal-pixel control, the $\sim 6.8\times$ scale factor was measured directly (twice, consistently) and used to compute `MarginV` from a desired literal-pixel target — documented in code as an explicitly *empirical* calibration, not a documented pixel mapping, with the exact reasoning and measurement method preserved so a future FFmpeg/libass version change can be recalibrated the same way.

Why architecturally correct: avoided introducing a new file-generation subsystem for a narrow, well-isolated positioning problem; the tradeoff (one recalibratable constant vs. real infrastructure) was made explicitly, with the “why” reasoning kept in the codebase rather than only in conversation history.

10.8 — Two unbounded FFmpeg inputs feeding one complex filtergraph

Root cause: adding a looped gradient-image input (`-loop 1`) as a *second* infinite-duration source into the same `-filter_complex` that already contained one infinite-duration source (looped background music, `-stream_loop -1`) caused FFmpeg to hang rather than error — `-shortest` does not reliably bound complex filtergraphs fed by multiple unbounded loop inputs, even though it correctly bounded a single one in the pre-existing code.

Investigation: caught by a hard sandbox timeout during testing, not a graceful failure — required isolating which of two nearly-simultaneous code changes (index fix, then this) was responsible, and reasoning about what was structurally new (two unbounded inputs at once, not one).

Final solution: explicitly bound the gradient input’s duration with `-t {video_duration}`, computed from already-known values (`total_duration + fixed_total`), rather than relying on `-shortest` to infer a stopping point.

Why architecturally correct: the fix addresses the actual new condition introduced (a second unbounded input), not a workaround; it required no change to how music’s own unbounded loop is handled, since that half of the graph was never the problem.

11. Important Design Decisions

All v1.0 decisions stand unchanged. Two more were made since:

- **Rejected:** keeping the subtitle gradient's positioning tied to the same fixed-offset approach used for the narration hero title's title/divider group. **Chosen instead:** a genuinely *centered* group for the question slide, after measuring (not assuming) that bottom-anchoring — correct for the hero title's small, tightly-fit region — left over 1000px of unintended empty space above short questions in the question slide's much larger available area. **Reason:** the same formula applied to two regions of very different scale relative to their content produced different, not just proportionally different, results; recognizing *when* a previously-correct pattern stops applying, rather than reapplying it by default, was the actual fix.
 - **Rejected:** treating the discovery that `original size` didn't work as a reason to build real `.ass`-file generation infrastructure. **Chosen instead:** an explicitly-documented empirical calibration constant. **Reason:** matches the project's standing bias toward the smallest change that resolves the specific evidenced problem — a real infrastructure investment was judged not justified by a single, narrow positioning issue.
-

12. Future Improvements (Not Yet Implemented)

Updated from the original list — **Mystery Archive as the second brand is removed (done, see Phase 11)**. Remaining and new items:

- **A generic, JSON-based StoryParser implementation**, offered as an adoptable option — now with two real brands’ worth of evidence about what a shared parsing format would actually need to support, not just a hypothetical design.
 - **A real segment_template enforcement mechanism** — still just documentation of intended shape today.
 - **Multi-platform publishing** (YouTube Shorts, TikTok).
 - **Cross-brand rate-limit coordination** for shared platform-level Groq/ElevenLabs accounts — the concrete trigger for eventually promoting `platform.py` into its own package. Still not needed; two brands running independently via separate GitHub Actions workflows hasn’t produced a real coordination need yet.
 - **A per-brand or fleet-wide dashboard** — the first real consumer of `brand.emoji / brand.parent`.
 - **Reconsidering per-brand GitHub Actions workflow duplication** — now more concrete, since two real, near-identical workflow files exist rather than one plus a hypothetical second.
 - **A generic Horror Lab/Mystery Lab parser abstraction** — deliberately still deferred; per the Phase 11 decision, revisit only if a *third* brand adopts the same line-format and demonstrates real, repeated maintenance cost, not before.
 - **A third brand** — the next real test of whether the “config, not branching” pattern (Phase 16) and the shared-vs-isolated code boundary (Phase 14’s test) continue to hold, or whether either needs revisiting under a genuinely new data point.
-

13. Lessons Learned

All four v1.0 lessons stand, and the work since v1.0 produced direct, reinforcing evidence for two of them specifically worth recording:

“A pattern, once found, tends to recur elsewhere until it is treated as a principle.” This recurred *again*, twice more (Phase 16): the “core silently defaults to shared/one-brand values” defect, previously found and fixed four separate times before v1.0, showed up a fifth and sixth time in the question-slide’s font size and text color — both hardcoded module constants nobody had thought to check because they weren’t part of the original architecture review’s checklist. The lesson holds even more strongly with more evidence: **a fixed instance of a named pattern doesn’t mean the pattern has been fully hunted down — it means the next occurrence is now recognizable faster.** Both fixes in Phase 16 took a fraction of the investigation time the original four did, specifically because the shape of the problem was already known.

“Claiming a fix was applied is not the same as verifying it was.” Reinforced directly during the subtitle work: the `original_size` fix was implemented, and only *because* its effect was actually measured against real rendered output — not assumed correct because it matched documentation — was it caught that the parameter did nothing at all. A plausible-sounding, documented, “should work” fix still requires the same evidence standard as any other change.

A new lesson, specific to this phase of work: distinguishing “this pattern applies again” from “this pattern applies here, but the numbers need to be re-derived, not reused.” The bottom-anchoring approach that worked correctly for the narration hero title was *structurally* the right idea for the question slide too — but blindly reapplying its exact relationship (fixed divider position, text bottom-anchored above it) without re-measuring against the question slide’s different available space produced a real, measured defect. The general shape of a solution transferring correctly doesn’t guarantee its specific parameters do.